Cybersecurity Challenges in Web Development

Technical Report

Student Name:	Omar Wasfy
Student ID:	39974723
Module Name:	LZIFC005M
Submission Date:	May 17, 2026

Contents

1 Introduction **2**

2 Common Web Vulnerabilities **2**

2.1 Cross-Site Scripting 2

3 Mitigation Techniques **3**

4 Real-World Scenario: The Equifax Breach **4**

5 Conclusion **5**

Glossary **5**

1 Introduction

Web application security is the practice of protecting websites, web services, databases, and users from attacks that target the way applications receive, process, store, and display data. A web application is not only a group of pages. It usually connects a browser, server-side code, APIs, authentication services, cookies, third-party libraries, and databases. Because these parts constantly exchange information, one weak point in input handling, access control, or configuration can create a serious risk.

Web security matters because everyday activities such as banking, shopping, studying, healthcare booking, and government services depend on web applications. If an attacker steals credentials, reads personal records, changes orders, or impersonates a user, the damage affects both individuals and organisations. The OWASP Top 10 describes common categories of web risk and shows that many attacks come from repeated development mistakes, including broken access control, injection flaws, and insecure design [4]. This report explains two common vulnerabilities, cross-site scripting and SQL injection, then discusses mitigation techniques and a real-world incident. Section 2 explains the vulnerabilities, while Table 1 summarises basic defences.

2 Common Web Vulnerabilities

2.1 Cross-Site Scripting

Cross-site scripting, or XSS, happens when a web application sends untrusted input back to a browser as active content. In simple terms, the attacker makes another user's browser run malicious JavaScript. This can happen when comments, profile fields, search terms, or error messages are displayed without proper output encoding. XSS is dangerous because the script runs inside the user's trusted session. Depending on the application, an attacker may steal session data, change page content, redirect the user to a fake login page, or perform actions as the victim.

Stored XSS is saved on the server, for example in a database comment field, and is triggered when another user opens the affected page. Reflected XSS is included in a request and immediately reflected in the response, such as a search page that prints the search term without escaping it. DOM-based XSS happens mainly in client-side code when JavaScript reads unsafe data from the URL and writes it into the document. Figure 1 shows the basic flow from malicious input to browser execution.

SQL injection occurs when an application builds a database query by directly mixing user input with SQL code. If the input is not handled safely, an attacker can change the meaning of the query. For example, a login form may expect a username and password, but malicious input can add SQL syntax that bypasses the intended check or exposes

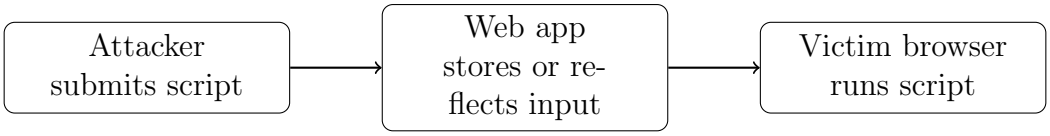

Figure 1: Simplified XSS flow from malicious input to browser execution.

extra rows. SQL injection can lead to unauthorised reading, editing, or deletion of data.

This vulnerability is serious because databases often hold the most valuable information in an application, including accounts, email addresses, password hashes, payment records, grades, or business data. OWASP recommends parameterised queries as a primary defence because they separate code from data, so user input is treated as a value rather than executable SQL [5]. Listing 1 shows a simplified unsafe query pattern.

Listing 1: Unsafe SQL query built from user input.

```
SELECT * FROM users
WHERE username = 'USER_INPUT'
AND password = 'PASSWORD_INPUT';
```

If raw input is inserted into this template, the attacker may add characters that change the condition. A safer design uses a prepared statement or an object-relational mapper that binds parameters securely.

3 Mitigation Techniques

Good web security depends on layers of defense rather than one single fix. Input validation checks whether data have the expected type, length, and format before the application processes them. For example, a quantity field should accept a reasonable number, not a script tag. However, validation alone is not enough because names, comments, and addresses may legitimately contain special characters. For XSS, output encoding is essential. Data should be encoded according to where it appears, such as HTML body, HTML attribute, JavaScript, or URL context. OWASP's XSS guidance explains that the correct defense depends on the output context [3].

For SQL injection, the strongest everyday mitigation is parameterised queries. With parameters, the database receives the SQL structure separately from the user-supplied values. Developers should also apply the least privilege to database accounts so that the web application does not connect with unnecessary administrator rights. Secure error handling is also important. Public error pages should not reveal SQL statements, file paths, stack traces, or database versions because this information helps attackers plan future attacks.

Security headers add another layer. The Content Security Policy can reduce the impact of XSS by restricting which scripts the browser is allowed to run. The secure and HttpOnly

cookie attributes can make session cookies harder to steal or misuse. Authentication and session management also need careful design, including strong password storage, session timeout, multi-factor authentication for sensitive accounts, and protection against cross-site request forgery where relevant. The NIST guide supports the general idea of selecting security controls to manage risk across systems [2].

Table 1: Examples of vulnerabilities and basic mitigations.

Issue	**Main risk**	**Basic mitigation**
XSS	Malicious script runs in a user's browser	Context-aware output encoding, safe templates, Content Security Policy
SQL injection	Attacker changes database queries	Parameterised queries, least-privilege database accounts, safe error handling
Weak session handling	Attacker impersonates a user	Secure cookies, session expiry, multi-factor authentication

As shown in Table 1, each control must match the weakness. A general phrase such as "sanitize input" is too vague unless the developer knows what data is allowed and where it will be used.

4 Real-World Scenario: The Equifax Breach

A major real-world example is the Equifax data breach, which became public in 2017. The incident is often discussed in web security because attackers exploited a known vulnerability in Apache Struts, a web application framework used by the company. The Federal Trade Commission's settlement page describes the breach as affecting personal information of many consumers and leading to a major settlement process [1]. The lesson is clear: web application security also includes dependency management and timely patching. A secure login form is not enough if the underlying framework has a known remote code execution flaw.

This case connects to the earlier sections because it shows why security must be part of the development lifecycle, not only a final test before release. Organizations need asset inventories, vulnerability monitoring, and clear responsibility for applying updates. The impact of a web vulnerability can also extend beyond technical downtime. It can cause legal costs, loss of trust, reputational damage, and long-term harm to users whose personal information is exposed.

5 Conclusion

Cybersecurity in web development is about building applications that safely handle untrusted input, protect user data, and continue to operate reliably. XSS and SQL injection are common examples because both come from the failure to separate trusted code from untrusted data. XSS allows an attacker-controlled script to run in a victim's browser, while SQL injection allows an attacker to change database commands. The main mitigations include context-aware output encoding, parameterised queries, least privilege, secure headers, careful session handling, and safe error messages. The Equifax breach shows that secure development also includes patching frameworks and managing dependencies. In general, web security is not a single tool or checklist item. It is a structured way of thinking about risk during design, coding, testing, deployment, and maintenance.

Glossary

XSS Cross-site scripting, where attacker-controlled script runs in another user's browser.

SQL Injection An attack where unsafe input changes the meaning of a database query.

CSP Content Security Policy, a browser security control that restricts allowed scripts and resources.

References

[1] Federal Trade Commission. *Equifax Data Breach Settlement*. Accessed 2026-05-15. 2019. URL: https://www.ftc.gov/enforcement/refunds/equifax-data-breach-settlement.

[2] National Institute of Standards and Technology. *Security and Privacy Controls for Information Systems and Organizations, SP 800-53 Revision 5*. Accessed 2026-05-15. 2020. URL: https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final.

[3] OWASP Foundation. *Cross Site Scripting Prevention Cheat Sheet*. Accessed 2026-05-15. 2024. URL: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html.

[4] OWASP Foundation. *OWASP Top 10: The Ten Most Critical Web Application Security Risks*. Accessed 2026-05-15. 2021. URL: https://owasp.org/www-project-top-ten/.

[5] OWASP Foundation. *SQL Injection Prevention Cheat Sheet*. Accessed 2026-05-15. 2024. URL: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html.